

Message Passing, RSA Encryption

Michael L. Littman

CS 0220 2021

June 25, 2021

Overview

The RSA Algorithm (8.8)
Sending Secrets

Sending messages

Motivations:

- ▶ Why do we send each other messages? Communication is a pretty human activity. Coordination is a practical application.
- ▶ Why would we want them to be secret? Competition. Gossiping. Surprise. Private information.
- ▶ Why might the message need to be encrypted? Message can be intercepted, stolen/broadcast by a third party, accidentally revealed like by being left on screen connected to projector. Communication channel is open.

Demo

Pick a card, any card.

Make use of prior agreement. In this case, via Email.

Encoding more messages

Generalizing this idea, we'll assume all messages are fixed-length bit strings.

Is that sufficiently general? Can we encode arbitrary messages this way?

The Alphabet

00000	space	01000	H	10000	P	11000	X
00001	A	01001	I	10001	Q	11001	Y
00010	B	01010	J	10010	R	11010	Z
00011	C	01011	K	10011	S	11011	.
00100	D	01100	L	10100	T	11100	!
00101	E	01101	M	10101	U	11101	?
00110	F	01110	N	10110	V	11110	,
00111	G	01111	O	10111	W	11111	@

One-time pad: Encryption

Alice and Bob share 60 random bits (the “one-time pad”) in advance:

```
10110 00011 00000 00110 10010 00010  
00000 11000 00110 01001 11111 11110
```

Alice: Wants to send a private message to Bob. She turns it into a sequence of 60 bits. She then computes the bitwise “xor” of her message and the one-time pad and transmits it:

```
00101 00010 10100 10011 00000 01100  
00000 11000 00110 01001 11111 11110
```

One-time pad: Decryption

Bob: Wants to read Alice's message.

```
00101 00010 10100 10011 00000 01100
00000 11000 00110 01001 11111 11110
```

How can he recover it? Bitwise “xor” with the one-time pad will undo the encryption operation.

```
encrypted line 1: 00101 00010 10100 10011 00000 01100
pad line 1:       10110 00011 00000 00110 10010 00010
xor line 1:       10011 00001 10100 10101 10010 01110
text line 1:      S A T U R N
encrypted line 2: 00000 11000 00110 01001 11111 11110
pad line 2:       00000 11000 00110 01001 11111 11110
xor line 2:       00000 00000 00000 00000 00000 00000
text line 2:      - - - - -
```


One-time pad: Cracking

Eve: Sees the encrypted message and wants to understand it. She doesn't have the one-time pad.

The encrypted message gives *no information* about the unencrypted message. All possible messages are *equally likely*.

Although, if one-time pad is reused, information is leaked.

Public key cryptography

The one-time pad is essential to the one-time pad scheme. How can Alice and Bob agree on the one-time pad if Eve is listening? Could send it encrypted with a one-time pad, but they'd use up n bits of pad to transmit n bits of pad, so that doesn't help.

The idea of public key cryptography is pretty wacky.

Bob wants to be able to receive secret messages. Bob creates a private key, which must remain secret. Bob also creates a public key, which is made public. Anyone, Alice say, who wants to send a secret message to Bob can encrypt it with Bob's public key. Only Bob can decrypt such messages (using his private key).

No other communication or agreements or secret emails are needed.

Beforehand

1. Bob generates two distinct (hundreds of digits long) primes, p and q and keeps them hidden.
2. Bob sets $n ::= pq$.
3. Bob selects an integer $e \in [1, n)$ such that $\gcd(e, (p-1)(q-1)) = 1$. The public key is the pair (e, n) .
4. Compute $d \in [1, n)$ such that $de \equiv 1 \pmod{(p-1)(q-1)}$. The private key is the pair (d, n) .

Time to send and receive

Encryption: Alice wants to send unencrypted message m . She computes and then sends the encrypted message $m^* = \text{rem}(m^e, n)$. (Uses e and n , which are public.)

Decryption: Bob receives m^* . He decrypts by computing $m' = \text{rem}((m^*)^d, n)$. (Uses d and n , where d is private.)

Decrypting works

Claim: $m' = m$.

Partial Proof (for case when m is relatively prime to n):

m'	$= \text{rem}((m^*)^d, n)$	defn m'
	$= \text{rem}((\text{rem}(m^e, n))^d, n)$	def of m^*
	$= \text{rem}((m^e)^d, n)$	“pre-mod” property
	$= \text{rem}(m^{ed}, n)$	exponentiation
	$= \text{rem}(m^{1+k(p-1)(q-1)}, n)$	selection of e
	$= \text{rem}(m^{1+k\phi(n)}, n)$	property of $\phi(pq)$
	$= \text{rem}(m \times (m^{\phi(n)})^k, n)$	rearrange factors
	$= \text{rem}(m, n)$	Euler’s thm
	$= m$	assuming $m < n$

Can handle $\text{gcd}(m, n) > 1$, too, but this argument doesn’t show why.

How compute?

1. How generate big primes p and q ? Factoring is hard, but can do (randomized) primality test quickly—Miller-Rabin. For d -digit prime, need about d tries to find one.
2. How multiply big primes pq to get n ? Standard multiplication algorithm scales well to big numbers.
3. How find number relatively prime to $(p-1)(q-1)$? Generate and test works again, but the chance of getting a hit at least as good and the test is much faster. Specifically, Euclid!
4. How compute $d = e^{-1} \bmod (p-1)(q-1)$? Pulvarize! (aka gcdcombo.)
5. How compute $m^* = m^e \bmod n$ or $m' = (m^*)^d \bmod n$?
Exponentiation by repeated squaring.

All of the steps are fast enough. You know all the algorithms (except Miller-Rabin).

Breaches

Private: p, q, d .

Public: n, e .

- ▶ If p revealed? Can get $q = n/p$. Can get d as $e^{-1} \bmod (p-1)(q-1)$.
- ▶ If q is revealed? Same as p .
- ▶ If d is revealed? Can get us p and q , though less clear how. But, can decrypt messages, so that's pretty bad.

We're pretty sure it's hard to get p and q from n . Factoring. We don't know if knowing e makes factoring easier, but no one has found a way to do it yet.

Other RSA uses

Encrypt with Bob's public key. Can only be decrypted with Bob's private key. Sending private messages to Bob.

Encrypt with Bob's private key. Can be decrypted with Bob's public key. Digital signature—only Bob can make such a message.

Encrypt with Bob's public key, then Alice's. Bob and Alice must work together to decrypt. Like an encrypted “and”.

Build your own!